

C PROGRAMMING FROM SCRATCH · EPISODE 06

Keywords in C

Reserved Words and Language Vocabulary

KEY POINTS

- Keywords are reserved by the language; words like int and return have built-in meanings.
- Two strict rules: you cannot invent new keywords, and variable/function names cannot collide with keywords.
- ANSI C (C89) defined exactly 32 classic keywords, making up almost all of C's core vocabulary.
- Attempting to name a variable after a keyword (e.g. int char;) causes a syntax error.

01 What Is a Keyword?

▶ VIDEO 00:51

- C reserves a small, fixed set of words for its grammar.
- Words like `int` (specifies integer type) and `return` (exits function and hands back a value) have built-in meanings.
- **Two strict rules**:
  1. You cannot invent new keywords of your own.
  2. You cannot use any keyword as the name of a variable or function.

02 The 32 Classic Keywords

▶ VIDEO 02:01

Finalized in ANSI C (C89), these 32 words make up the foundational vocabulary of C:

CATEGORY	KEYWORDS
Data Types & Modifiers	`char`, `int`, `float`, `double`, `short`, `long`, `signed`, `unsigned`, `void`
Control Flow	`if`, `else`, `switch`, `case`, `default`, `while`, `for`, `do`, `break`, `continue`, `return`, `goto`
Custom Types	`struct`, `union`, `enum`, `typedef`
Storage & Qualifiers	`auto`, `register`, `static`, `extern`, `const`, `volatile`
Operator	`sizeof`

> NOTE
>
> You do not need to memorize all 32 at once; we will practice each one naturally as we write real programs.

03 Modern Standards

▶ VIDEO 03:31

- **C99**: Added `inline`, `restrict`, and `_Bool` (the leading underscore and uppercase `B` avoided breaking older code).
- **C11 & C23**: Added keywords like `_Static_assert`, and officially standardized `bool`, `true`, and `false`.
- The complete list can always be checked on **cppreference.com/w/c/keyword**.

04 Naming Collisions

▶ VIDEO 04:47

Variable names are user-defined **identifiers**. When an identifier collides with a keyword, the build fails:

variable-naming.c — C SOURCE

```c
#include <stdio.h>

int main(void)
{
    int a;
    a = 10;
    return 0;
}
```

KEY IDEA **Valid Identifier**

`int a;` builds cleanly: `int` is the type keyword, and `a` is a valid identifier.

keyword-identifier.c — C SOURCE

```c
/* This file does not compile on purpose:
 * keywords cannot be variable names */
#include <stdio.h>

int main(void)
{
    int char;
    return 0;
}
```

Output · Show output from: Build

```
1>hello.c
1>hello.c(5,9): error C2632: 'int' followed by 'char' is illegal
========== Build: 0 succeeded, 1 failed ==========
```

KEY IDEA **Why Collisions Fail**

If `char` were allowed as a variable name, the compiler could not tell whether `char` refers to the character type or to your variable. Keeping keywords reserved prevents grammatical ambiguity.

CODE 2 files on GitHub github.com/abitclearer/c-programming-from-scratch/tree/main/06-keywords-in-c